Bangladesh University of Engineering and Technology

Course no: EEE 212

Course title: Numerical Technique Laboratory

Project Title: Curl, Divergence and Gradient of a field.

Students' name: 1.Moshiur Rahman

2.Nilanjona Majumder Medha

3.Shanta Sarker

ID: **1.2006099**

2.2006100

3.2006103

Section: B

Lab group:B-2

Submitted to: Suzit Hasan Nayem

Meftaul Haque

Date of submission: 28/02/2023

Documentation

Introduction:

In vector calculus, we study the calculus of vector fields, which are functions that assign a vector to each point in space. Curl, divergence, and gradient are three important operators in vector calculus that help us to analyze vector fields.

Curl, divergence, and gradient have numerous applications in various fields of study, such as physics, engineering, and computer graphics. For example, in fluid dynamics, the divergence and curl of a velocity field can be used to study fluid flow, while in computer graphics, the gradient of a scalar field can be used to create realistic textures.

In MATLAB, though these operators can be easily implemented using built-in functions such as "gradient", "divergence", and "curl", we used numerical methods to calculate these operators and analyze vector fields and their plots.

Objectives:

- ✓ To provide a comprehensive understanding of the concepts of curl, divergence, and gradient in vector calculus.
- ✓ To explain the methodology used to implement curl, divergence, and gradient in MATLAB and provide sample code for reference.
- ✓ To present the results of calculations using MATLAB and analyze their significance through their plots using simple code and Graphical User Interface (GUI)

Motivations:

- ➢ To gain a deeper understanding of vector calculus and its importance in various fields of study.
- ➢ To learn how to use MATLAB to analyze and calculate vector fields using curl, divergence, and gradient.
- ➢ To develop the ability to interpret and analyze data generated by MATLAB calculations.
- ➢ To apply the concepts of curl, divergence, and gradient to real-world problems and explore their practical applications.

Overall, the objectives of this report are to provide a comprehensive understanding of the concepts of curl, divergence, and gradient, their practical applications, and

how to implement them in MATLAB. The motivations for this report include gaining a deeper understanding of vector calculus, developing skills in using MATLAB, and exploring the real-world applications of these concepts.

Theoretical background:

Concept:

The gradient is a vector operator that represents the rate of change of a scalar field. It is defined as the vector field of the partial derivatives of a scalar field with respect to each of the coordinate axes.

For a real-valued function f(x,y,z) on R^3, the gradient ∇f(x,y,z) is a vector-valued function on R^3, that is, its value at a point (x,y,z) is the vector

$$\mathrm{Grad(f)} = \nabla f(x,y,z) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}\right) = \frac{\partial f}{\partial x} i + \frac{\partial f}{\partial y} j + \frac{\partial f}{\partial z} k$$

in R^3, where each of the partial derivatives is evaluated at the point (x,y,z)

The symbol ∇ is "applied" to a real-valued function f to produce a vector ∇f.

Where,

$$\nabla = \frac{\partial}{\partial x} i + \frac{\partial}{\partial y} j + \frac{\partial}{\partial z} k$$

Here, the symbols $\frac{\partial}{\partial x}, \frac{\partial}{\partial y}, \frac{\partial}{\partial z}$ are to be thought of as "partial derivative operators" that will get "applied" to a real-valued function, say f(x,y,z).

The divergence is a scalar operator that represents the extent to which a vector field flows outward from a given point. It is defined as the sum of the partial derivatives of a vector field with respect to each of the coordinate axes.**.** Divergence denotes only the magnitude of change and so, it is a scalar quantity. It does not have a direction.

The divergence is expressed as div f as ∇·f, since for a vector field
f(x,y,z)=f1(x,y,z)i+f2(x,y,z)j+f3(x,y,z)k

, the dot product of f with ∇

∇·f=(∂/∂xi+∂/∂yj+∂/∂zk)·(f1(x,y,z)i+f2(x,y,z)j+f3(x,y,z)k)

$=(\frac{\partial}{\partial x})f1+(\frac{\partial}{\partial y})f2+(\frac{\partial}{\partial z})f3$

$=\frac{\partial}{\partial x}f1+\frac{\partial}{\partial y}f2+\frac{\partial}{\partial z}f3$

=div(f)

If the divergence is positive at a point, then the vector field is flowing out of that point, while if it is negative, the field is flowing into that point. A zero divergence indicates that the field is neither flowing out nor into that point.

The curl is a vector operator that represents the rotation or circulation of a vector field. It is defined as the vector product of the del operator and a vector field.when the flow is counter-clockwise, curl is considered to be positive and when it is clock-wise, curl is negative.

We can also write curl f in terms of ∇, namely as ∇×f

, since for a vector field f(x,y,z)=P(x,y,z)i+Q(x,y,z)j+R(x,y,z)k

$$\begin{aligned}
\nabla \times \mathbf{f} &= \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ \dfrac{\partial}{\partial x} & \dfrac{\partial}{\partial y} & \dfrac{\partial}{\partial z} \\ P(x,y,z) & Q(x,y,z) & R(x,y,z) \end{vmatrix} \\
&= \left(\frac{\partial R}{\partial y} - \frac{\partial Q}{\partial z}\right)\mathbf{i} - \left(\frac{\partial R}{\partial x} - \frac{\partial P}{\partial z}\right)\mathbf{j} + \left(\frac{\partial Q}{\partial x} - \frac{\partial P}{\partial y}\right)\mathbf{k} \\
&= \left(\frac{\partial R}{\partial y} - \frac{\partial Q}{\partial z}\right)\mathbf{i} + \left(\frac{\partial P}{\partial z} - \frac{\partial R}{\partial x}\right)\mathbf{j} + \left(\frac{\partial Q}{\partial x} - \frac{\partial P}{\partial y}\right)\mathbf{k} \\
&= \operatorname{curl} \mathbf{f}
\end{aligned}$$

The curl of F at a point measures the tendency of the vector field to rotate about that point. If the curl is zero at a point, then the field is irrotational, while a nonzero curl indicates that the field is rotational.

Modified code as instructed (using numerical analysis)

Created functions:

```
function dx = Dx(f,x,y,z)
delx = 0.0001;
dx = (f(x + delx,y,z) - f(x - delx,y,z))/(2*delx);
end
function dy = Dy(g,x,y,z)
dely = 0.0001;
dy = (g(x,y+dely,z) - g(x,y-dely,z))/(2*dely);
end
function dz = Dz(h,x,y,z)
delz = 0.0001;
dz = (h(x,y,z+delz) - h(x,y,z-delz))/(2*delz);
end
```

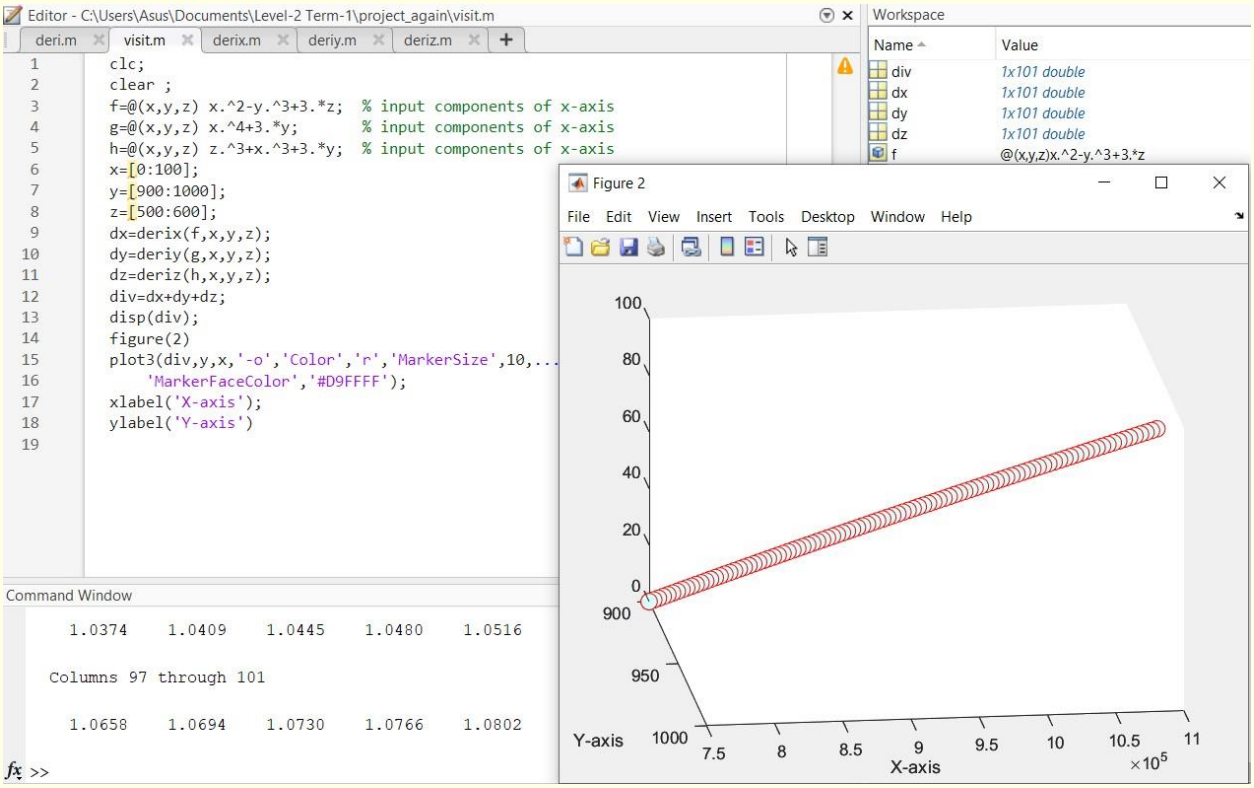
Divergence:

Main code:

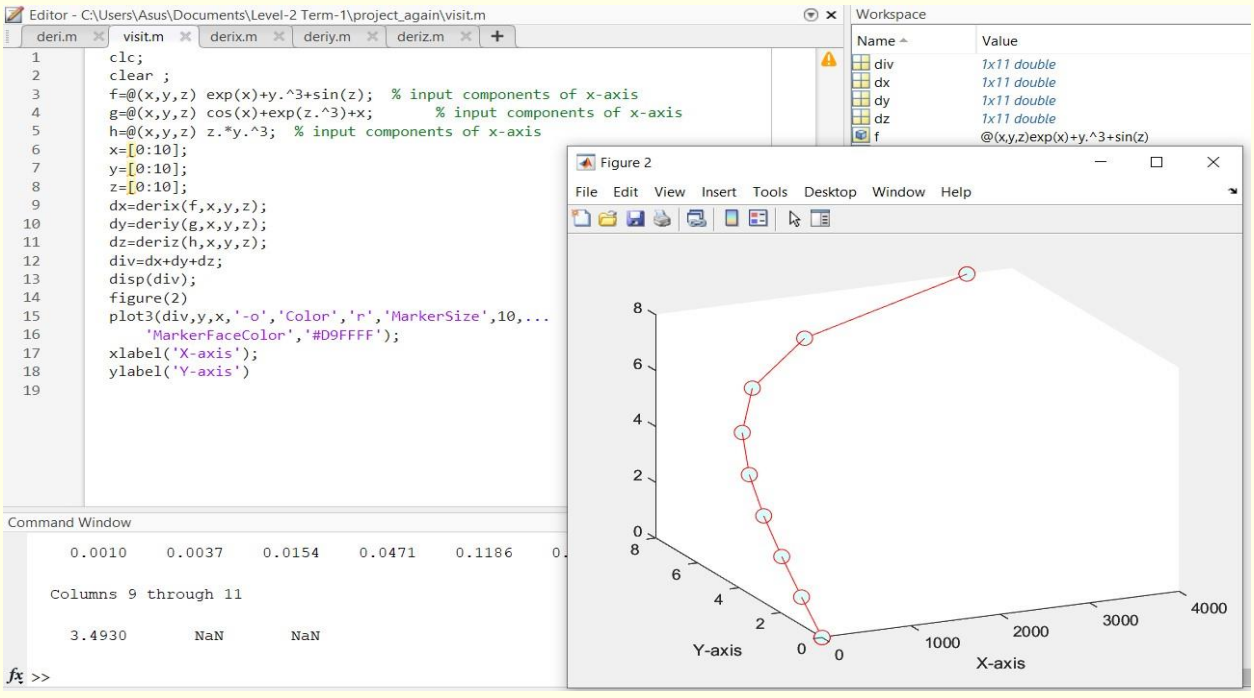
```
clc;
clear ;
f=@(x,y,z) x.^2-y.^3+3.*z;  % input components of x-axis
g=@(x,y,z) x.^4+3.*y;       % input components of x-axis
h=@(x,y,z) z.^3+x.^3+3.*y;  % input components of x-axis
x=[0:100];
y=[900:1000];
z=[500:600];
dx=derix(f,x,y,z);
dy=deriy(g,x,y,z);
dz=deriz(h,x,y,z);
div=dx+dy+dz;
disp(div);
figure(2)
plot3(div,y,x,'-o','Color','r','MarkerSize',10,...
    'MarkerFaceColor','#D9FFFF');
xlabel('X-axis');
ylabel('Y-axis')
```

Outputs:

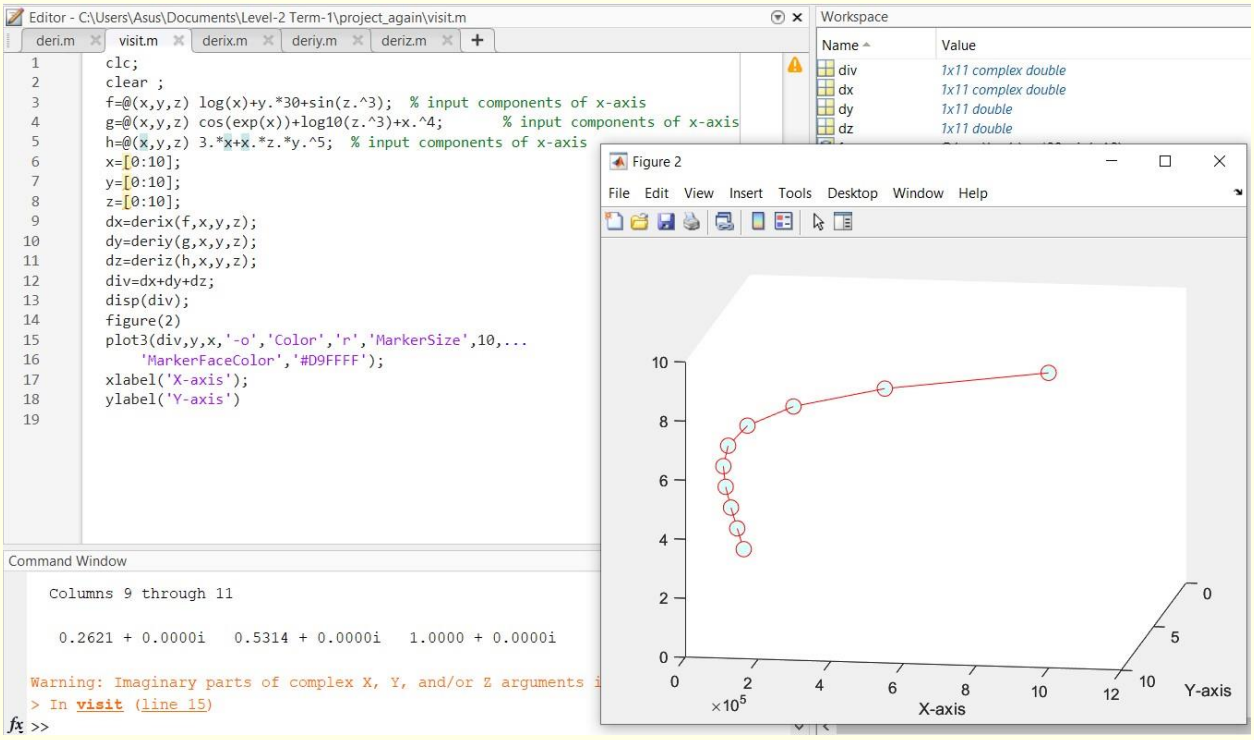
Case 1:



Case 2:



Case 3:

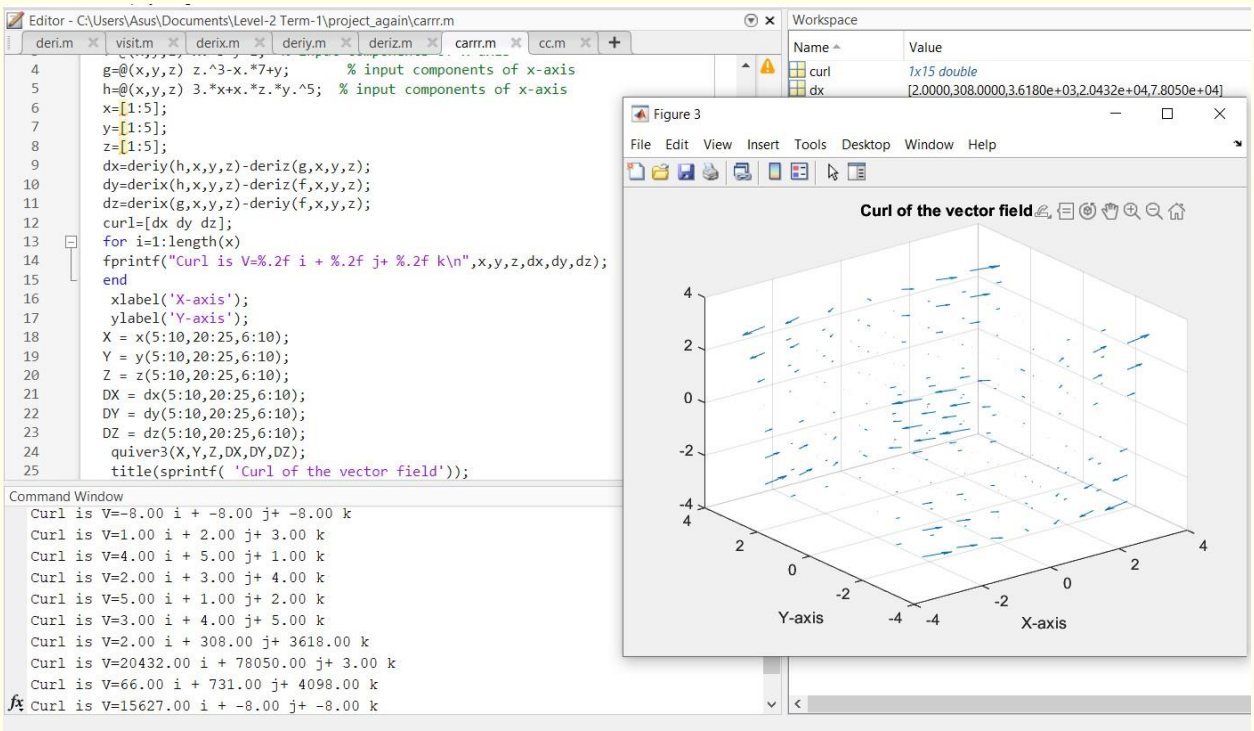


Curl: Main code:

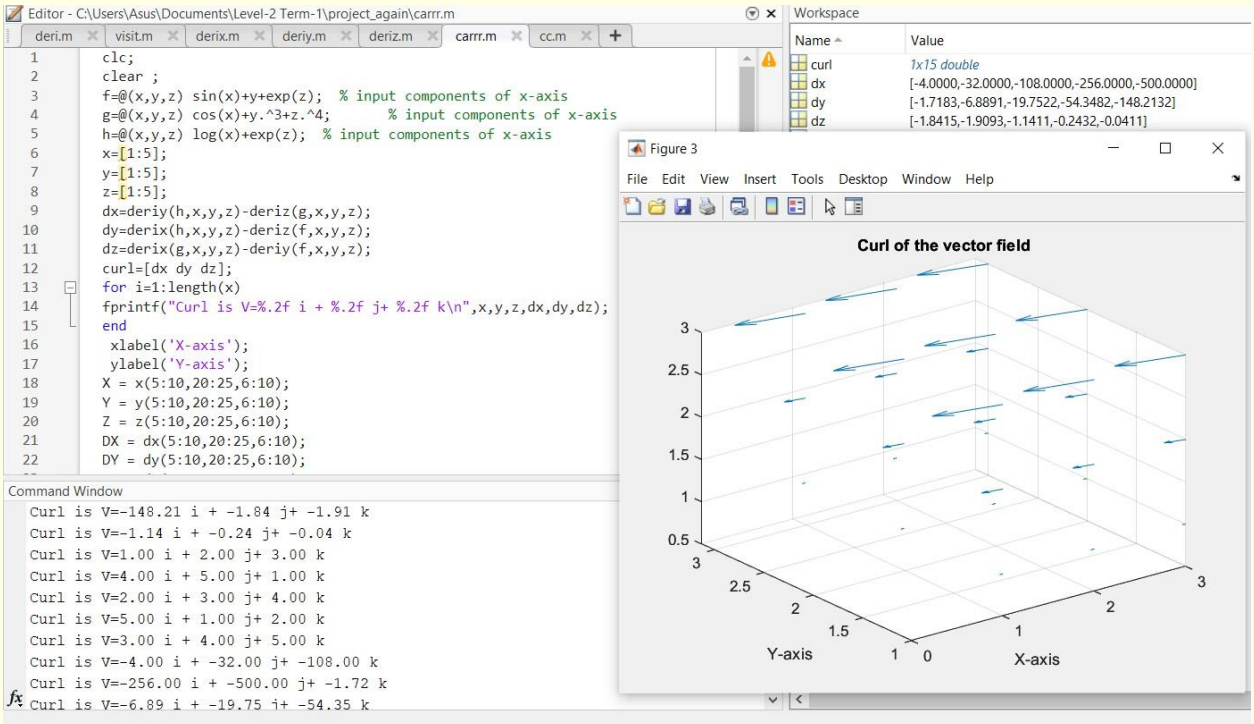
```
clc;
clear ;
f=@(x,y,z) x.^2-y.^3;  % input components of x-axis
g=@(x,y,z) x.^3-y.^4+z.^4;       % input components of
x-axis
h=@(x,y,z) log10(x)+exp(z)-3.*y;  % input components of
x-axis
x=[1:5];
y=[1:5];
z=[1:5];
dx=deriy(h,x,y,z)-deriz(g,x,y,z);
dy=derix(h,x,y,z)-deriz(f,x,y,z);
dz=derix(g,x,y,z)-deriy(f,x,y,z);
curl=[dx dy dz];
for i=1:length(x)
fprintf("Curl is V=%.2f i + %.2f j+ %.2f
k\n",x,y,z,dx,dy,dz);
end
 xlabel('X-axis');
 ylabel('Y-axis');
X = x(5:10,20:25,6:10);
Y = y(5:10,20:25,6:10);
Z = z(5:10,20:25,6:10);
DX = dx(5:10,20:25,6:10);
DY = dy(5:10,20:25,6:10);
DZ = dz(5:10,20:25,6:10);
 quiver3(X,Y,Z,DX,DY,DZ);
 title(sprintf( 'Curl of the vector field'));
```

Output:

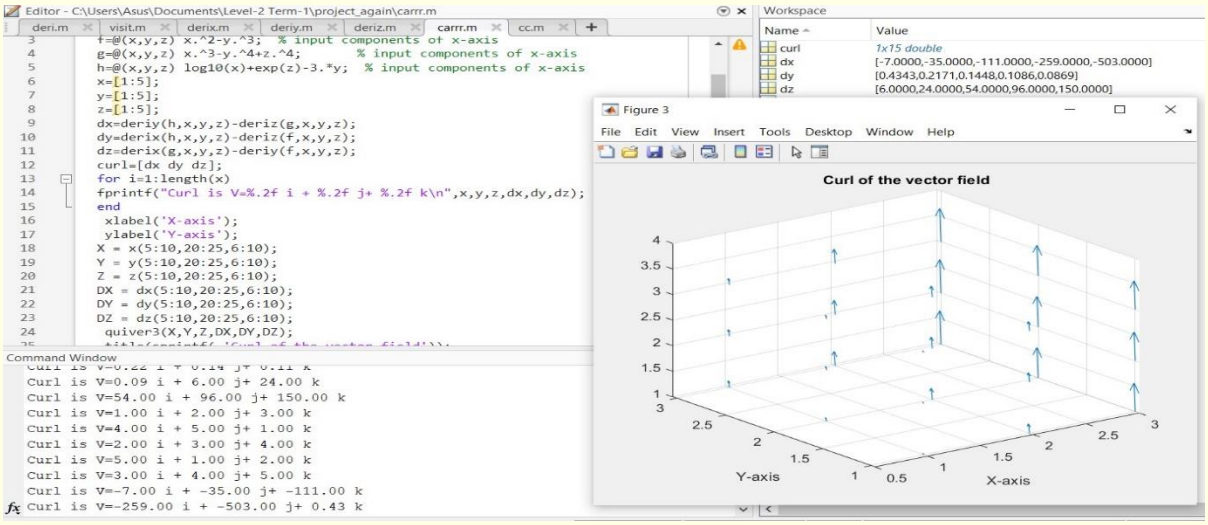
Case 1:



Case 2:



Case 3:



Function value show Code:

```
clc;
clear;
close all;
%x,y,z are the symbolic variables to use
x= sym('x');
y= sym('y');
z= sym('z');
%Gradient computation
scalar_func=input('Introduce a scalar field to obtain GRADIENT:');
fprintf("Gradient:")
value=[diff(scalar_func,x),diff(scalar_func,y),diff(scalar_func,z)]
%to input the components x,y,z of the vector%a=x component of the vector
a=input('Input the x component of the vector:');%b=y component of the vector
b=input('Input the y component of the vector:'); %c=z component of the vector
c=input('Input the z component of the vector:');%The vector is output on the command
disp('The vector is:');
F=[a,b,c];
disp(['F',' = '])
disp(F);
%Partial derivatives of each component
dx=diff(a,x);
dx1=diff(a,y);
dx2=diff(a,z);
dy=diff(b,y);
dy1=diff(b,x);
dy2=diff(b,z);
dz=diff(c,z);
dz1=diff(c,x);
dz2=diff(c,y);
%The divergence is the sum of the previous values
disp('Divergence:');
div=dx+dy+dz
%Components of the curl vector(determinant)
comx=diff(c,y)-diff(b,z);
comy=diff(a,z)-diff(c,x);
comz=diff(b,x)-diff(a,y);
%The curl is the vector formed by the components:
disp(['Curl:']);
rot=[comx,comy,comz]
```

Output:-

Command Window

```
Introduce a scalar field to obtain GRADIENT:3*x*y*z
Gradient:
value =

[3*y*z, 3*x*z, 3*x*y]

Input the x component of the vector:3*x^2*y^2
Input the y component of the vector:10*y^2*z^3
Input the z component of the vector:2*x*y*z^2
The vector is:
F =
[3*x^2*y^2, 10*y^2*z^3, 2*x*y*z^2]

Divergence:

div =

6*x*y^2 + 20*y*z^3 + 4*x*y*z

Curl:

rot =

[2*x*z^2 - 30*y^2*z^2, -2*y*z^2, -6*x^2*y]

fx >>
```

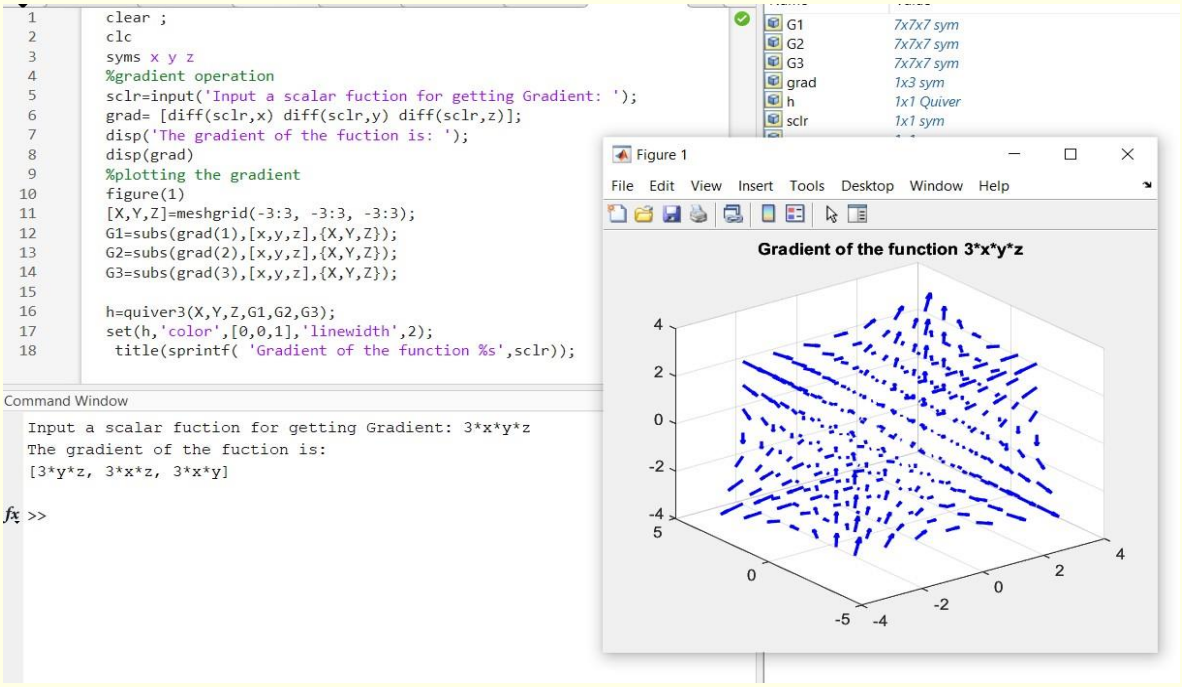
Plotting :

1. Gradient:
   Code:

```
clear ;
clc
syms x y z
%gradient operation
sclr=input('Input a scalar fuction for getting Gradient:
');
grad= [diff(sclr,x) diff(sclr,y) diff(sclr,z)];
disp('The gradient of the fuction is: ');
disp(grad);
%plotting the gradient
figure(1)
[X,Y,Z]=meshgrid(-3:3, -3:3, -3:3);
G1=subs(grad(1),[x,y,z],{X,Y,Z});
G2=subs(grad(2),[x,y,z],{X,Y,Z});
G3=subs(grad(3),[x,y,z],{X,Y,Z});

h=quiver3(X,Y,Z,G1,G2,G3);
set(h,'color',[0,0,1],'linewidth',2);
 title(sprintf( 'Gradient of the function %s',sclr));
```

Output:

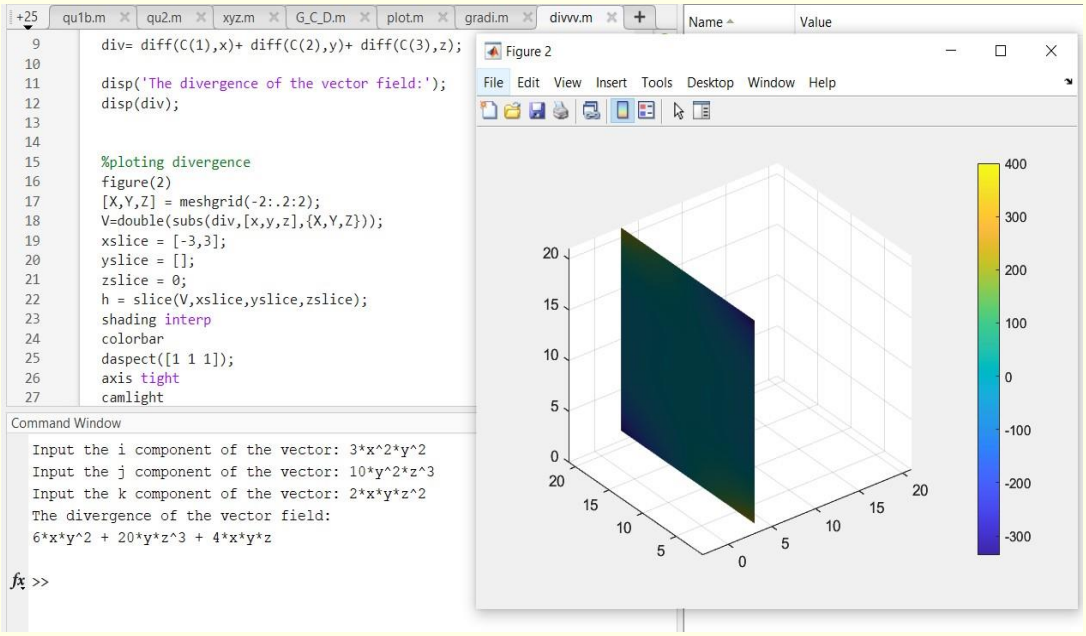


2. Divergence:

```
clear ;
clc
syms x y z i j k
%divergence operation
C(1)=input('Input the i component of the vector: ');
C(2)=input('Input the j component of the vector: ');
C(3)=input('Input the k component of the vector: ');
div= diff(C(1),x)+ diff(C(2),y)+ diff(C(3),z);
disp('The divergence of the vector field:');
disp(div);

%ploting divergence
figure(2)
[X,Y,Z] = meshgrid(-2:.2:2);
V=double(subs(div,[x,y,z],{X,Y,Z}));
xslice = [-3,3];
yslice = [];
zslice = 0;
h = slice(V,xslice,yslice,zslice);
shading interp
colorbar
daspect([1 1 1]);
axis tight
camlight
```

Output:



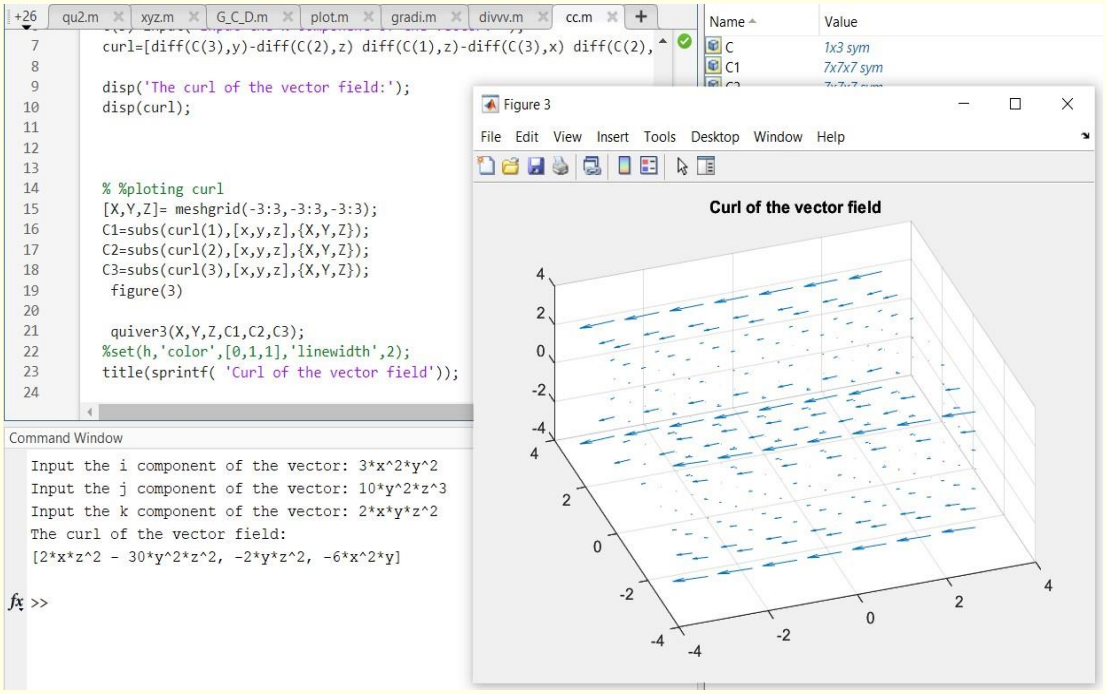
3.Curl:

Code:

```
clear ;
clc
syms x y z i j k
C(1)=input('Input the i component of the vector: ');
C(2)=input('Input the j component of the vector: ');
C(3)=input('Input the k component of the vector: ');
curl=[diff(C(3),y)-diff(C(2),z) diff(C(1),z)-diff(C(3),x)
diff(C(2),x)-diff(C(1),y)];
disp('The curl of the vector field:');
disp(curl);

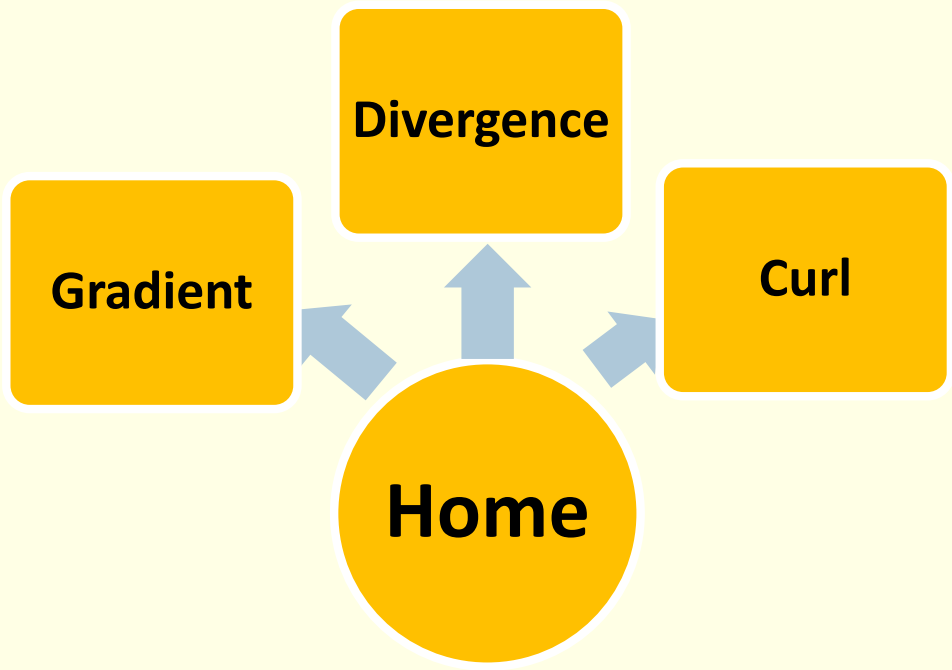
% %ploting curl
[X,Y,Z]= meshgrid(-3:3,-3:3,-3:3);
C1=subs(curl(1),[x,y,z],{X,Y,Z});
C2=subs(curl(2),[x,y,z],{X,Y,Z});
C3=subs(curl(3),[x,y,z],{X,Y,Z});
 figure(3)
 quiver3(X,Y,Z,C1,C2,C3);
%set(h,'color',[0,1,1],'linewidth',2);
title(sprintf( 'Curl of the vector field'));
```

Output:



Gradient,Curl,Divergence plot using MATLAB app designer

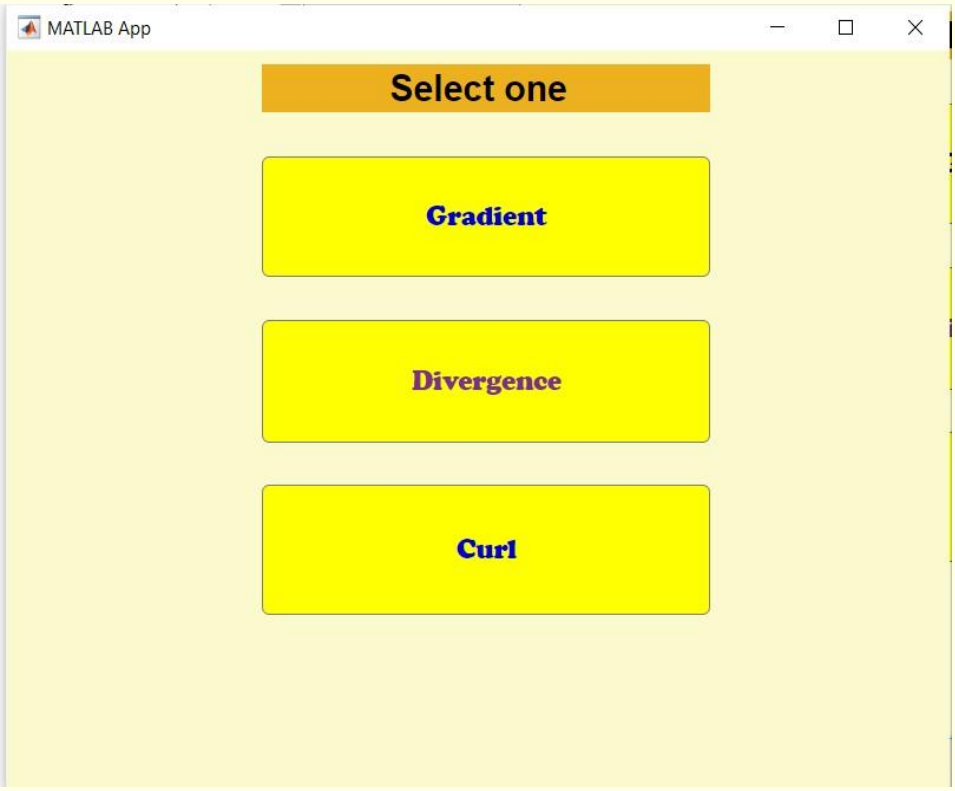
Methodology:



Algorithm:

1. The app will execute the HOME app, then according to the user's command, it will go further GRADIENT/DIVERGENCE/CURL.
2. Then in the input box, the function will be written in arithmetic order defined by MATLAB.
3. Then pressing CALCULATE button, the numerical values of the gradient/curl/divergence will be shown in the particular box (Here we used the 'diff' (a built in function in MATLAB) which is used as a partial differentiator of the functions).
4. Then we need to set the limit of x,y,z to plot the 3-D graph of the gradient/divergence/curl (For this we used some build in functions like **meshgrid, subs, quiver3,slice, colourbar** etc to find the plot that will execute some information about the scalar or vector field we took as input function)

Home page:



(The user has to select any of the options according to his choice)

- If he selects Gradient, then he needs to input a scalar function
- If he selects Divergence or Curl, then he needs to input a vector function .

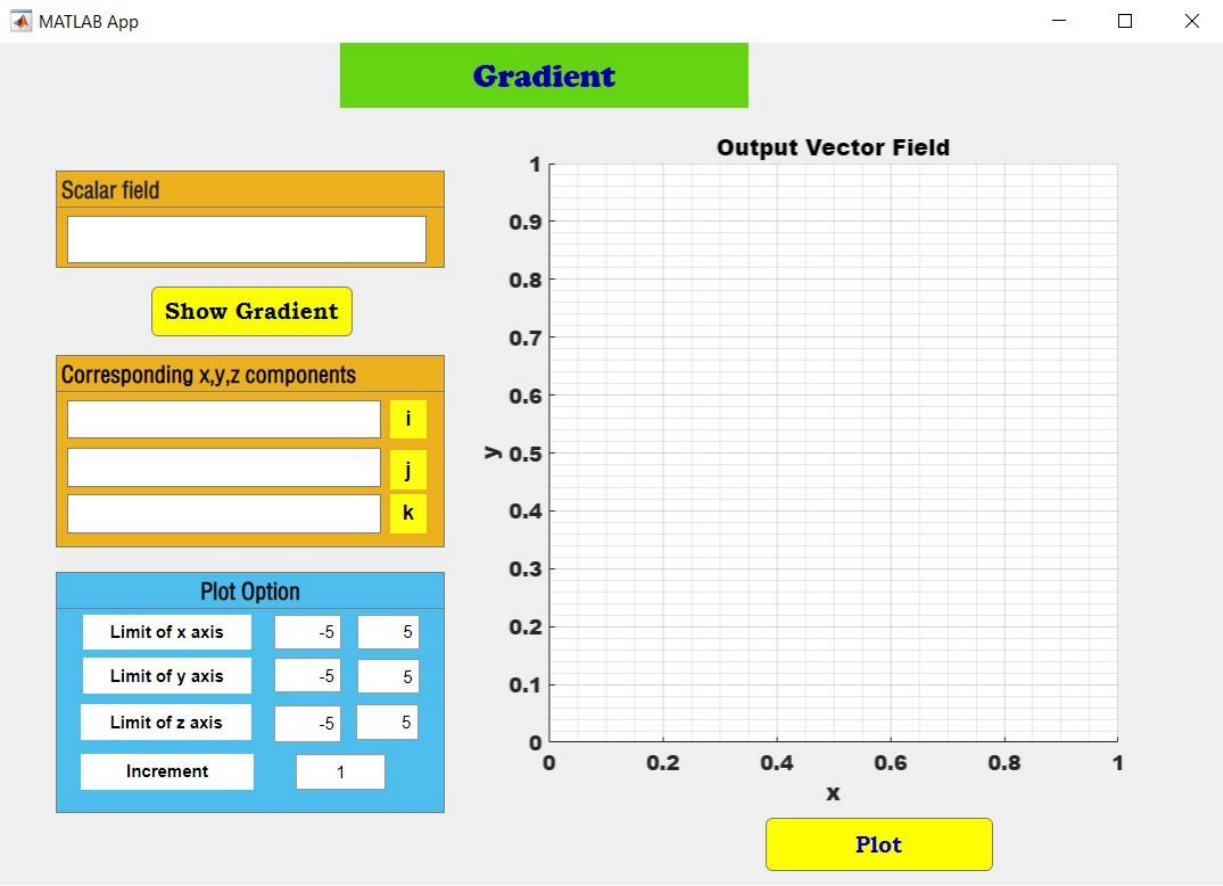
Callbacks for the buttons:

```
methods (Access = private)
        % Button pushed function: CurlButton
        function CurlButtonPushed(app, event)
            Home.app.callingapp=cUrL;
        end
        % Button pushed function: GradientButton
        function GradientButtonPushed(app, event)
            Home.app.callingapp=gRaD;
        end

        % Button pushed function: DivergenceButton
        function DivergenceButtonPushed(app, event)
            Home.app.callingapp=dIv;
        end
    end
```

Gradient interface:

(before input):



Callbacks :

```
methods (Access = private)

        function G=gradient(F)
            G=[(diff(F,x)) (diff(F,y)) (diff(F,z))];
        end
    end


    % Callbacks that handle component events
    methods (Access = private)

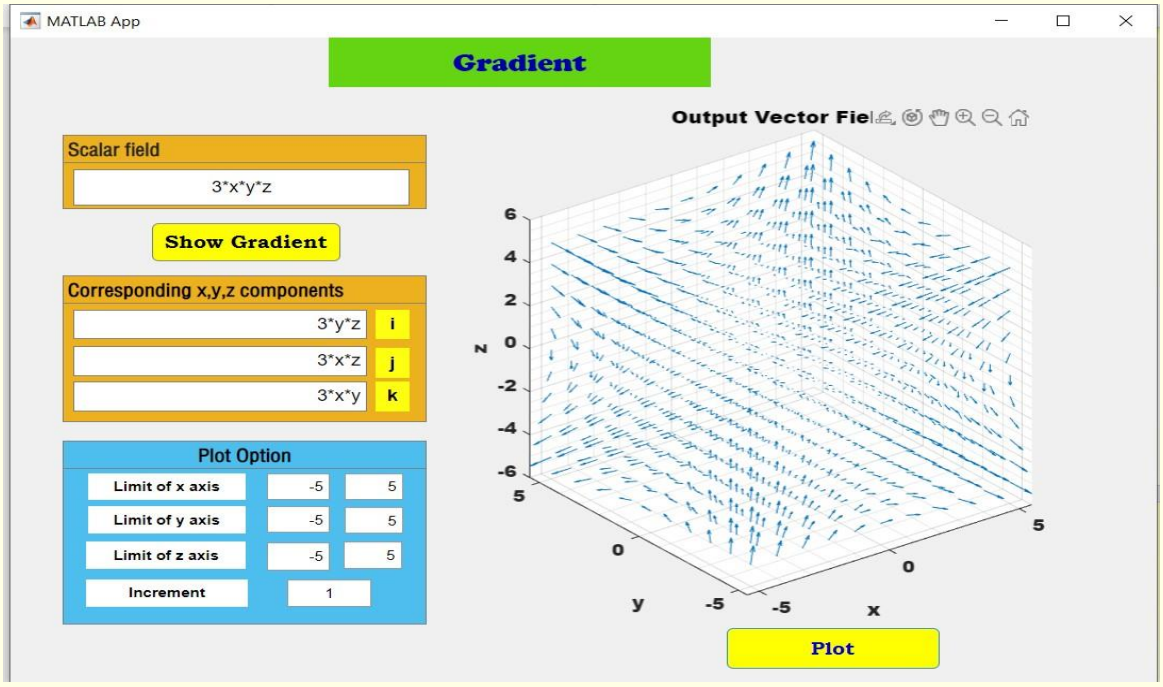
        % Button pushed function: ShowGradientButton
        function ShowGradientButtonPushed(app, event)
            syms x y z
            G=gradient(str2sym(app.EditField.Value));
            app.EditField_2.Value=string(G(1));
            app.EditField_3.Value=string(G(2));
            app.EditField_4.Value=string(G(3));
        end

        % Button pushed function: PlotButton
        function PlotButtonPushed(app, event)
            syms x y z;
            G=gradient(str2sym(app.EditField.Value));
            xL=app.EditField_5.Value;
            xU=app.EditField_6.Value;
            yL=app.EditField_7.Value;
            yU=app.EditField_8.Value;
            zL=app.EditField_9.Value;
            zU=app.EditField_10.Value;
            s=app.EditField_11.Value;
            [X,Y,Z]=meshgrid(xL:s:xU,yL:s:yU,zL:s:zU);
            u=subs(G(1),{x,y,z},{X,Y,Z});
            v=subs(G(2),{x,y,z},{X,Y,Z});
            w=subs(G(3),{x,y,z},{X,Y,Z});
            quiver3(app.UIAxes,X,Y,Z,u,v,w);
        end

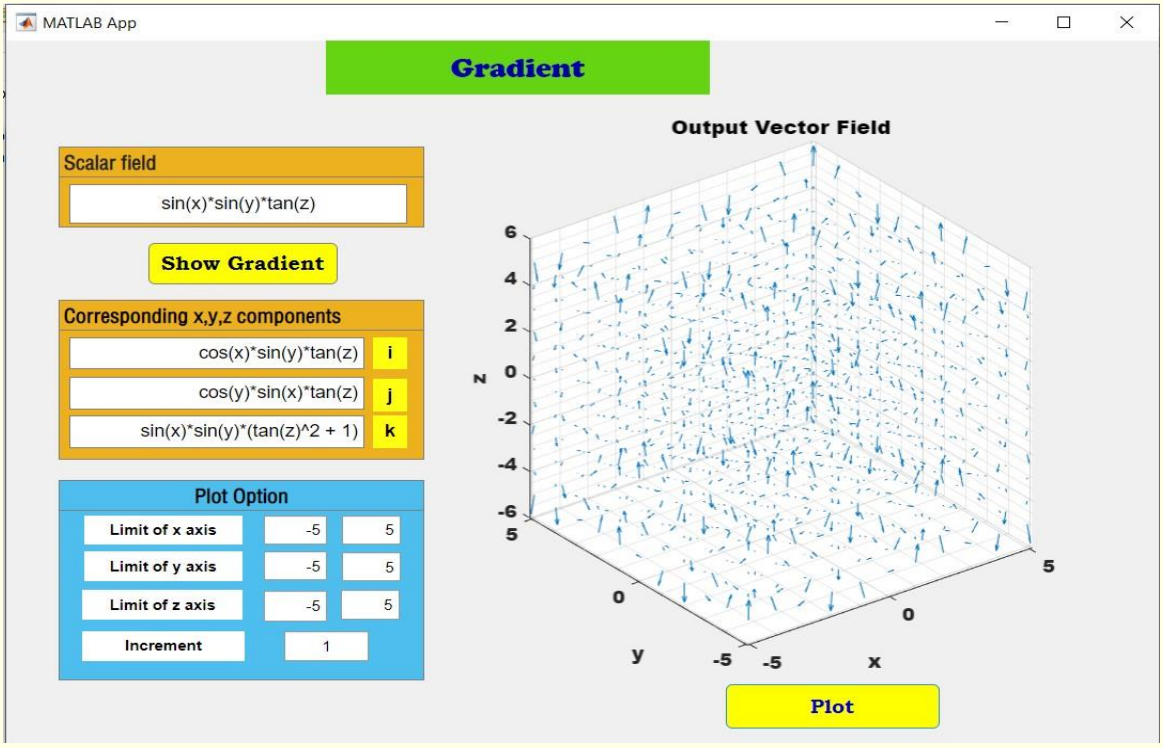
        % Callback function
        function HomeButtonPushed(app, event)
            Gradient.app.callingapp=Home;
        end
    end
```

Sample results:

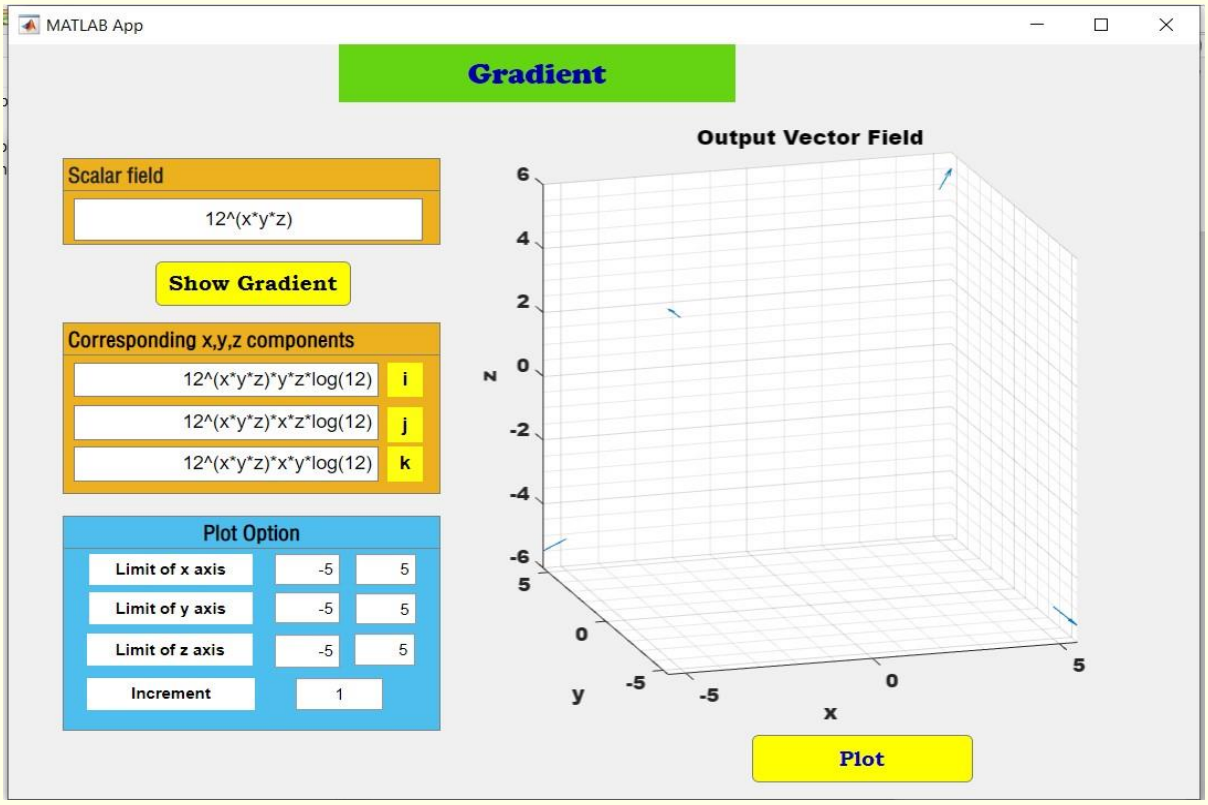
Case 1:



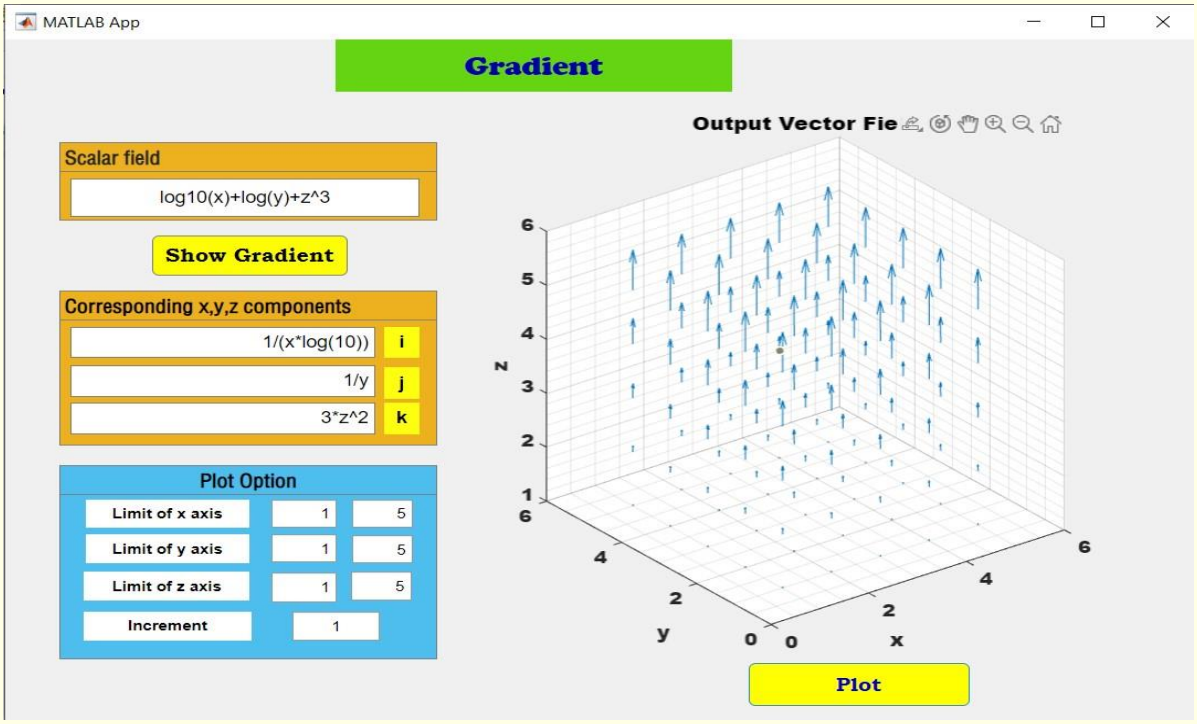
Case 2:



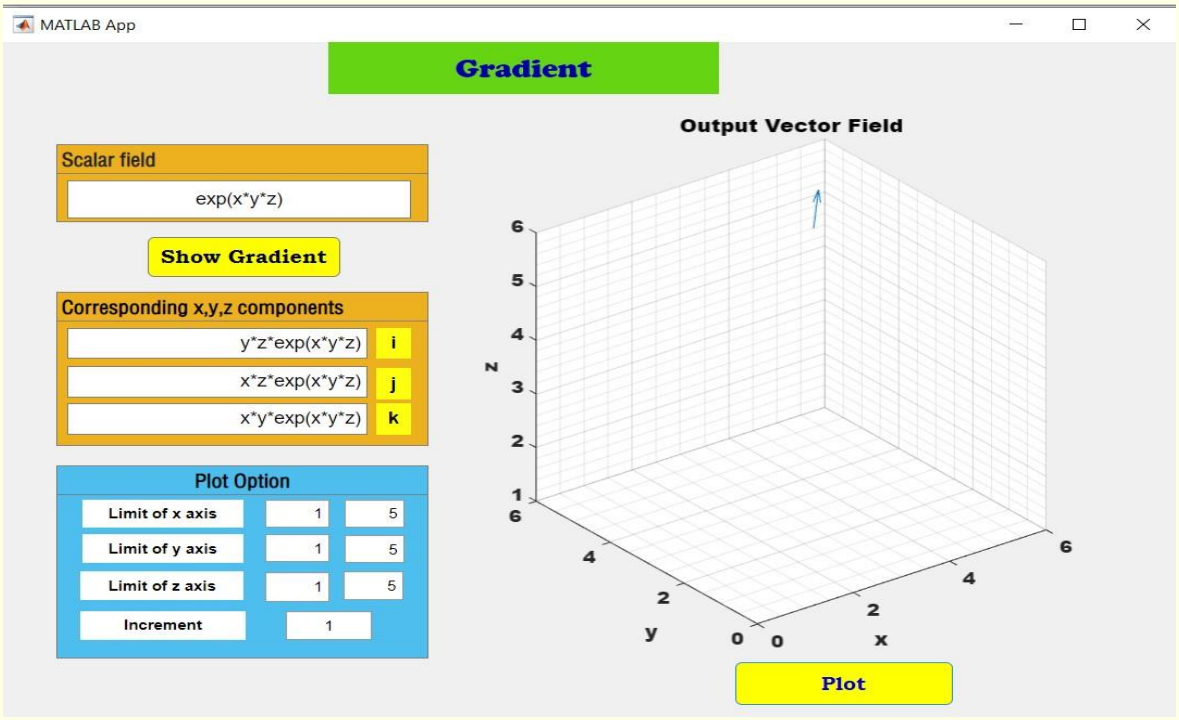
Case 3:


MATLAB App
Gradient
Scalar field
12^(x*y*z)
Show Gradient
Corresponding x,y,z components
12^(x*y*z)*y*z*log(12)
i
12^(x*y*z)*x*z*log(12)
j
12^(x*y*z)*x*y*log(12)
k
Plot Option
Limit of x axis
-5
5
Limit of y axis
-5
5
Limit of z axis
-5
5
Increment
1
Output Vector Field
Plot

Case 4:

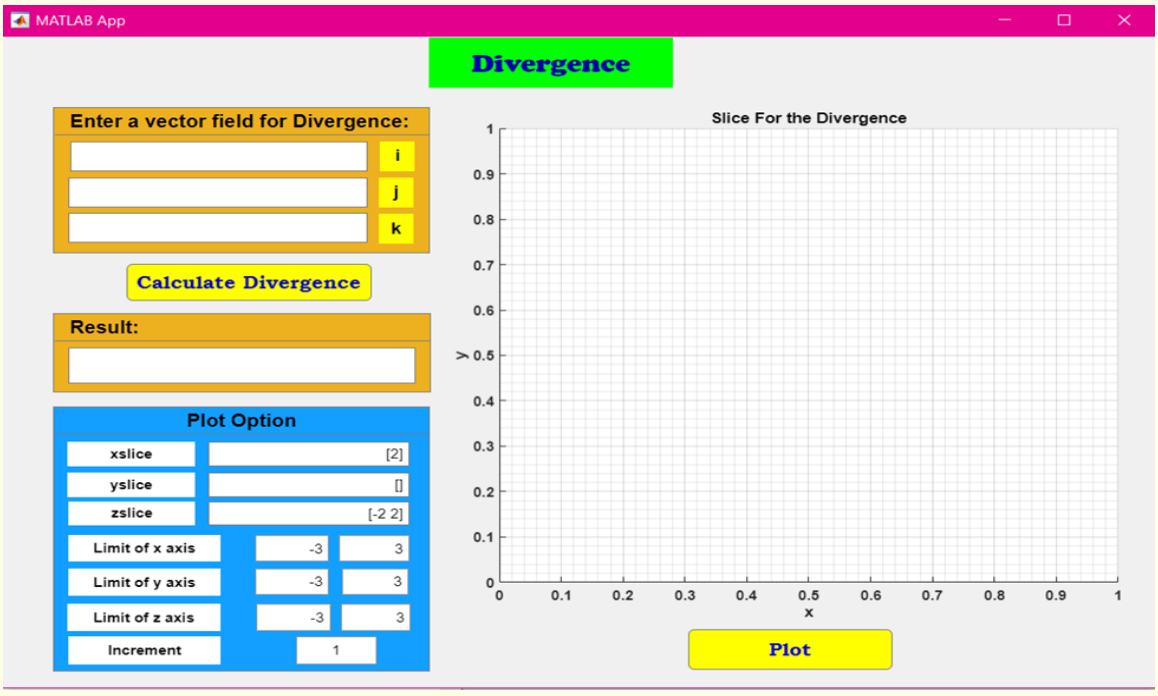

MATLAB App
Gradient
Scalar field
log10(x)+log(y)+z^3
Show Gradient
Corresponding x,y,z components
1/(x*log(10))
i
1/y
j
3*z^2
k
Plot Option
Limit of x axis
1
5
Limit of y axis
1
5
Limit of z axis
1
5
Increment
1
Output Vector Fie
Plot

Case 5:



Divergence interface:

(before input):



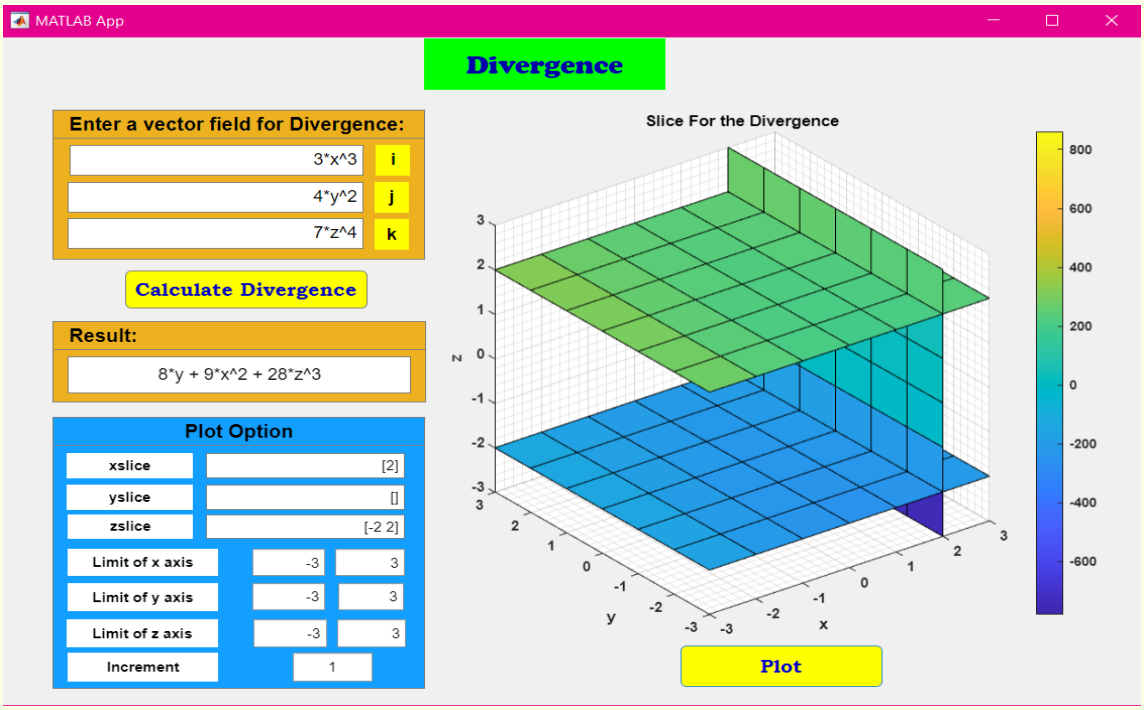
Callbacks:

```
methods (Access = private)
        function D = divergence(app)
            D=diff(app(1),x)+diff(app(2),y)+diff(app(3),z);
        end
    end
% Callbacks that handle component events
    methods (Access = private)

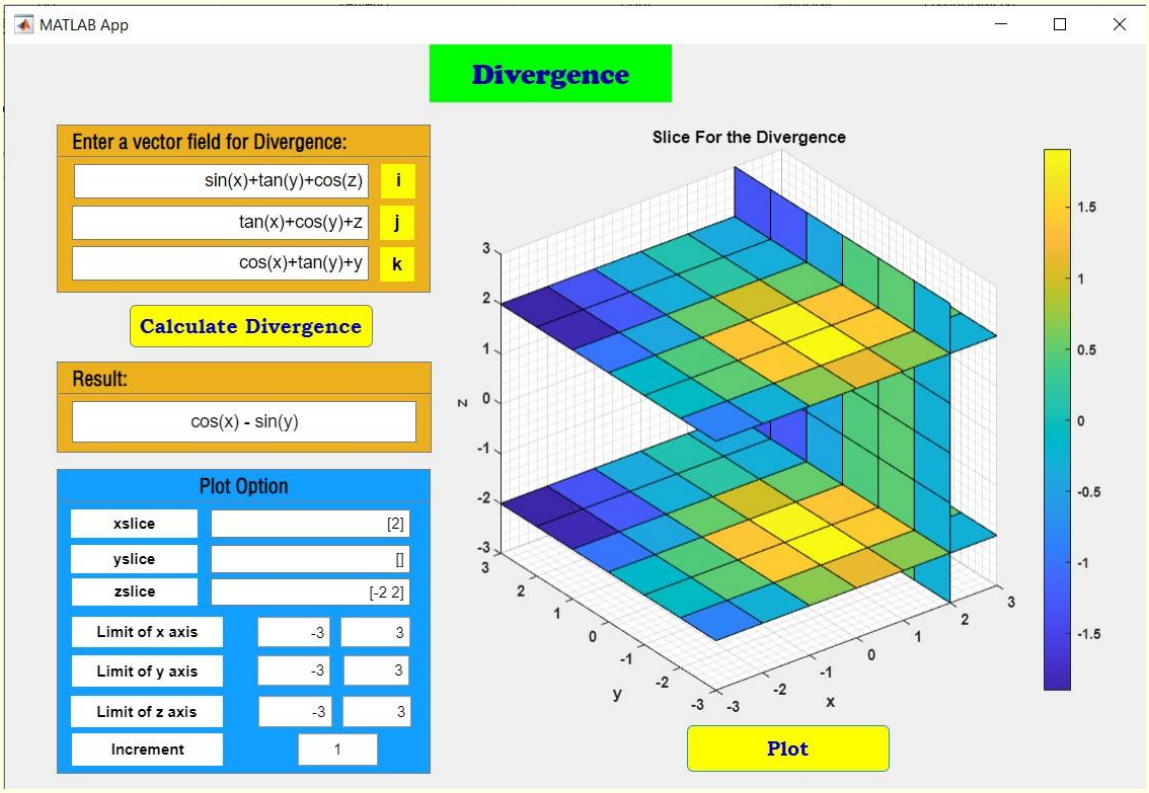
        % Button pushed function: CalculateDivergenceButton
        function CalculateDivergenceButtonPushed(app, event)
            syms x y z
            A(1)=str2sym(app.EditField.Value);
            A(2)=str2sym(app.EditField_2.Value);
            A(3)=str2sym(app.EditField_3.Value);
            D=divergence(A);
            app.EditField_4.Value=string(D);
        end
        % Callback function
        function HomeButtonPushed(app, event)
            Divergence.app.callingapp=Home;
        end
        % Button pushed function: PlotButton
        function PlotButtonPushed(app, event)
            syms x y z
            A(1)=str2sym(app.EditField.Value);
            A(2)=str2sym(app.EditField_2.Value);
            A(3)=str2sym(app.EditField_3.Value);
            D=divergence(A);
            xL=app.EditField_5.Value;
            xU=app.EditField_6.Value;
            yL=app.EditField_7.Value;
            yU=app.EditField_8.Value;
            zL=app.EditField_9.Value;
            zU=app.EditField_10.Value;
            s=app.EditField_11.Value;
            [X,Y,Z]=meshgrid(xL:s:xU,yL:s:yU,zL:s:zU);
            V=double(subs(D,{x,y,z},{X,Y,Z}));
            xslice=str2num(app.xsliceEditField.Value);
            yslice=str2num(app.ysliceEditField.Value);
            zslice=str2num(app.zsliceEditField.Value);
            slice(app.UIAxes,X,Y,Z,V,xslice,yslice,zslice);
            colorbar(app.UIAxes);
        end
    end
```

Sample results:

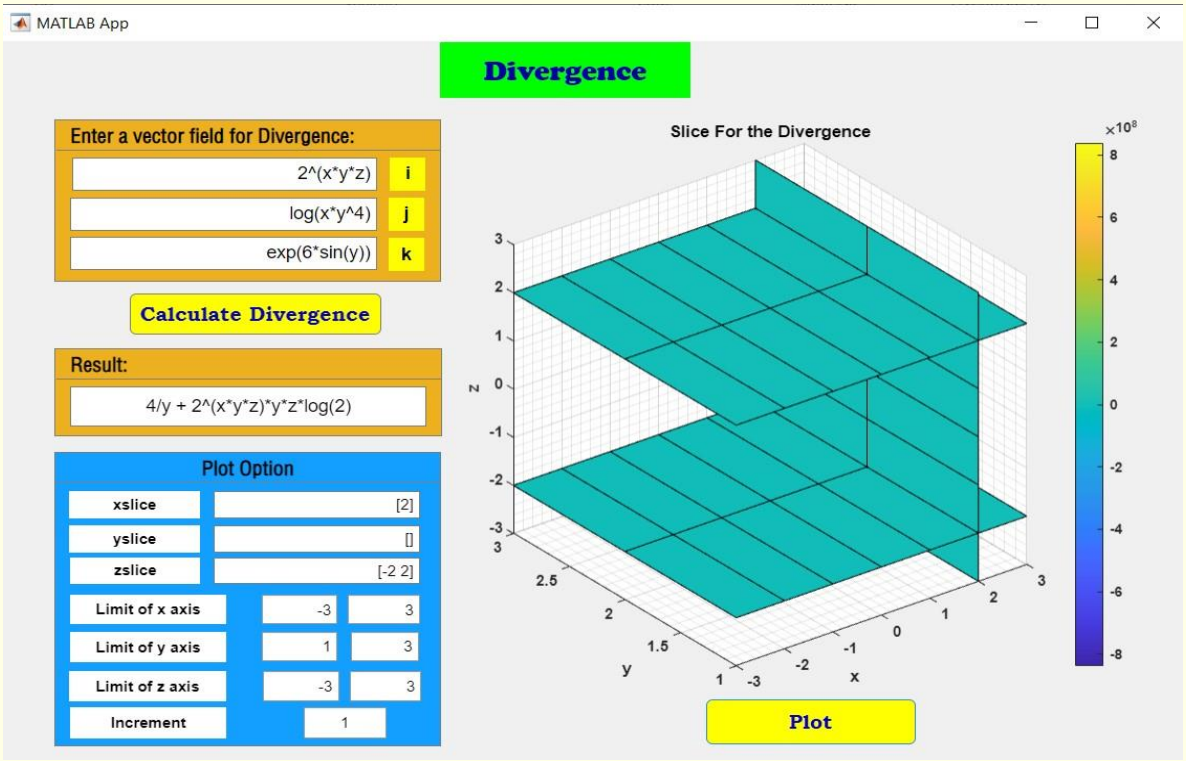
Case 1:



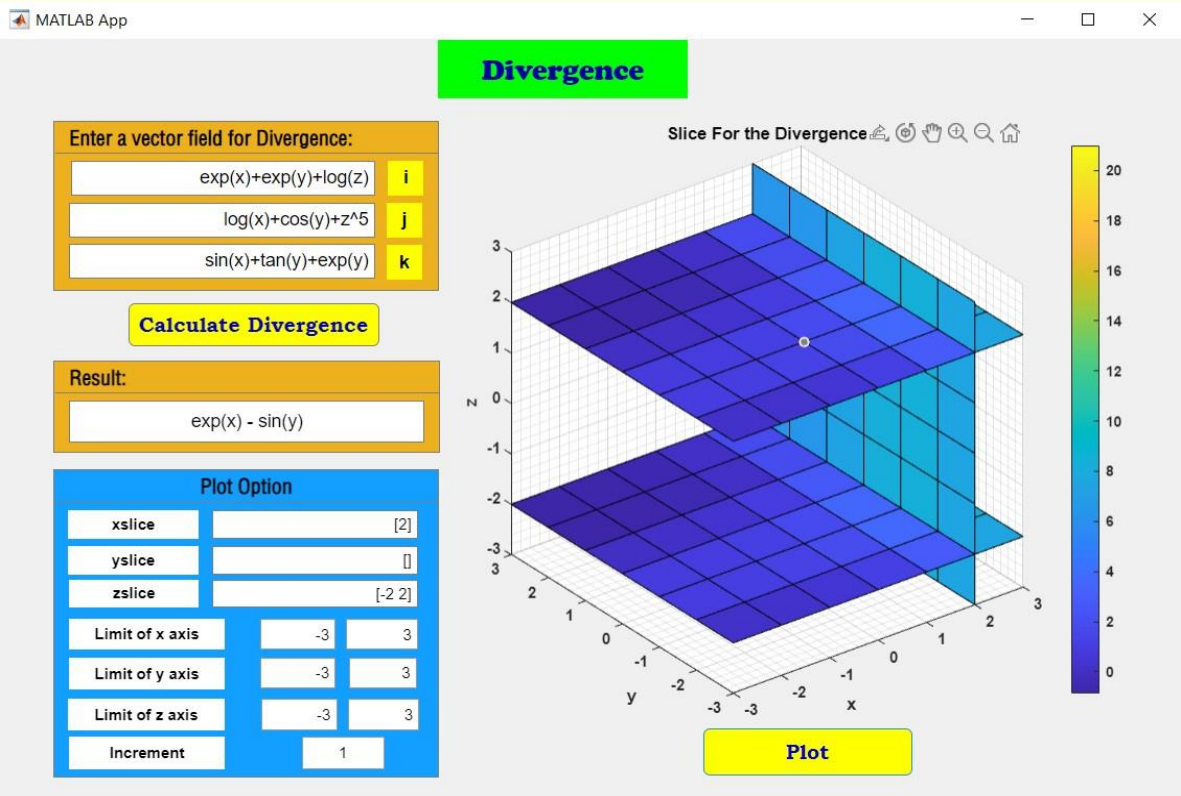
Case 2:



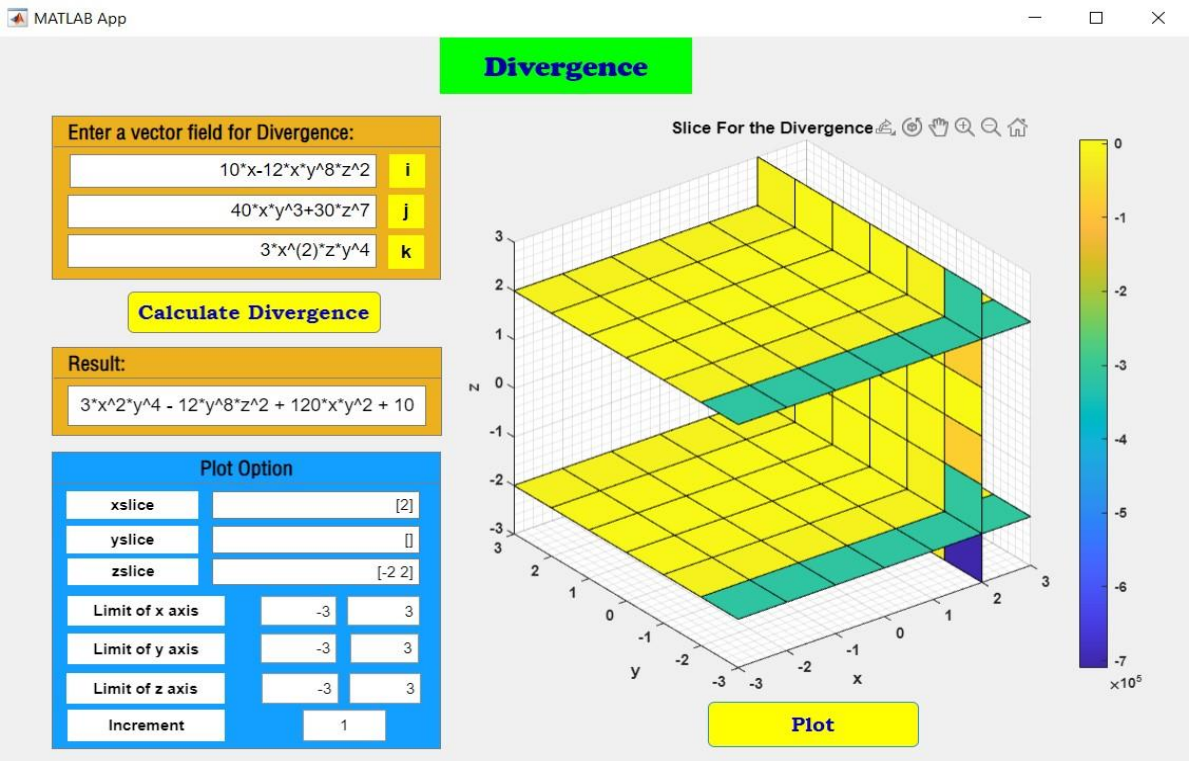
Case 3:


MATLAB App
Divergence
Enter a vector field for Divergence:
2^(x*y*z)
i
log(x*y^4)
j
exp(6*sin(y))
k
Calculate Divergence
Result:
4/y + 2^(x*y*z)*y*z*log(2)
Plot Option
xslice
[2]
yslice
[]
zslice
[-2 2]
Limit of x axis
-3
3
Limit of y axis
1
3
Limit of z axis
-3
3
Increment
1
Slice For the Divergence
x10^8
Plot

Case 4:

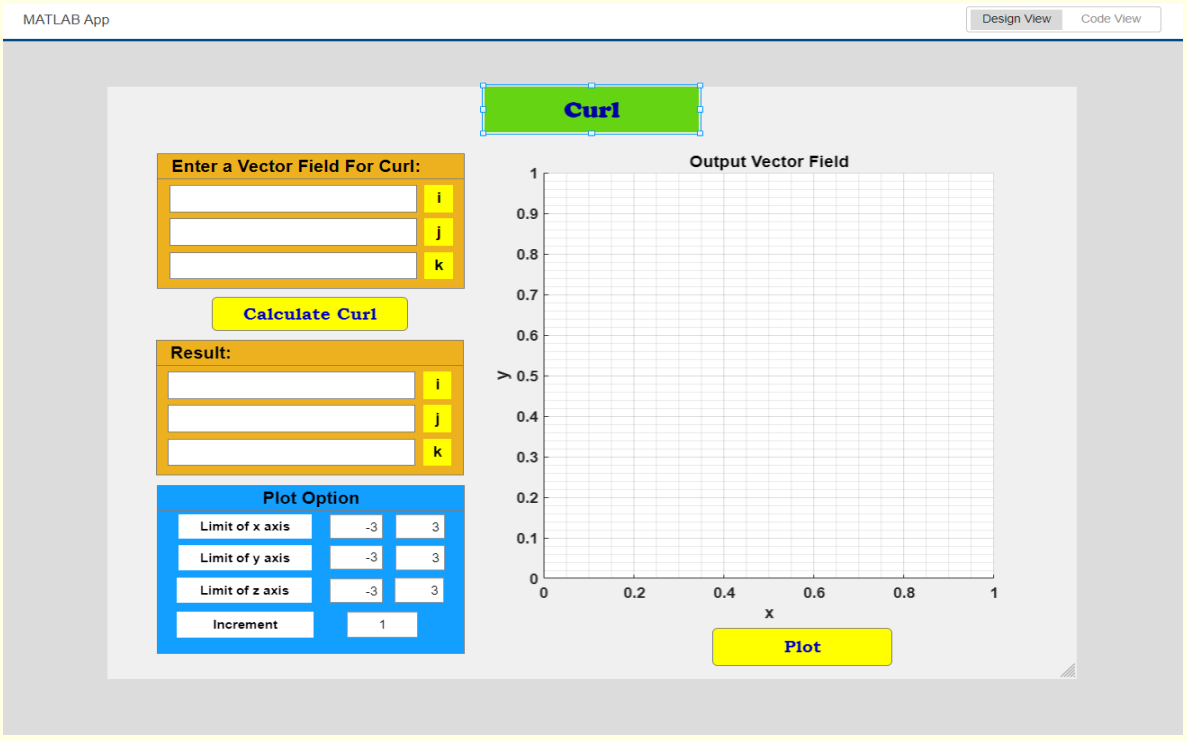

MATLAB App
Divergence
Enter a vector field for Divergence:
exp(x)+exp(y)+log(z)
i
log(x)+cos(y)+z^5
j
sin(x)+tan(y)+exp(y)
k
Calculate Divergence
Result:
exp(x) - sin(y)
Plot Option
xslice
[2]
yslice
[]
zslice
[-2 2]
Limit of x axis
-3
3
Limit of y axis
-3
3
Limit of z axis
-3
3
Increment
1
Slice For the Divergence
Plot

Case 5:



Divergence interface:

(before input):



Callbacks:

```
methods (Access = private)

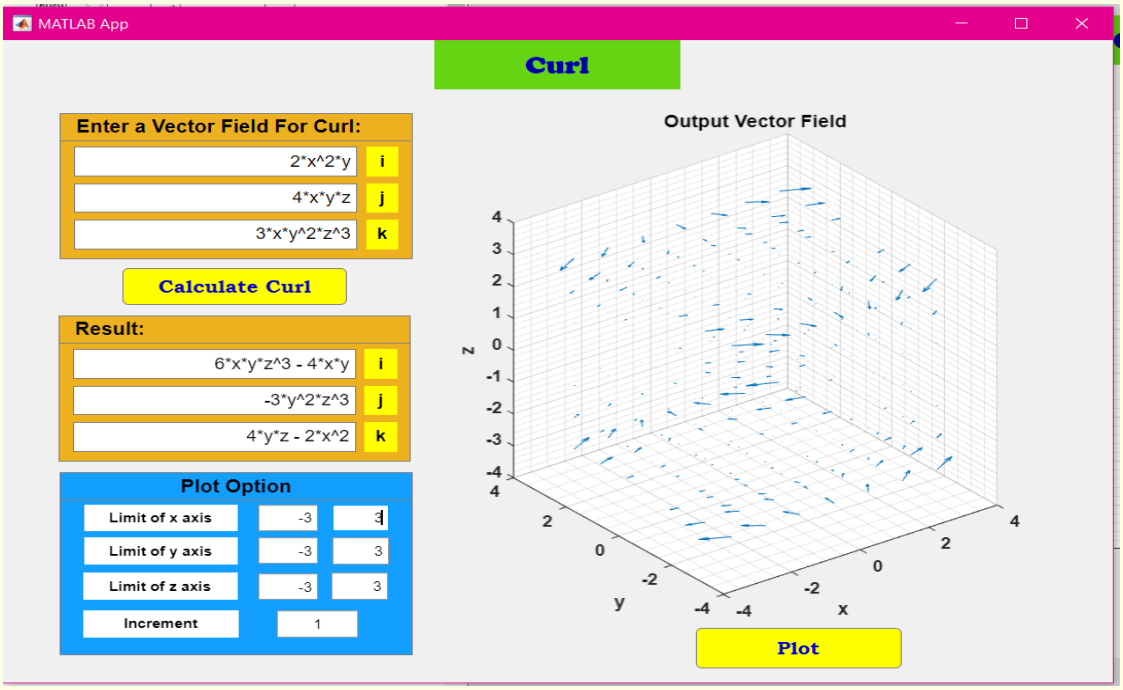
        % Button pushed function: CalculateCurlButton
        function CalculateCurlButtonPushed(app, event)
            syms x y z
            A(1)=str2sym(app.EditField.Value);
            A(2)=str2sym(app.EditField_2.Value);
            A(3)=str2sym(app.EditField_3.Value);
            C=[(diff(A(3),y)-diff(A(2),z)) -(diff(A(3),x)-
diff(A(1),z)) (diff(A(2),x)-diff(A(1),y))];
            app.EditField_4.Value=string(C(1));
            app.EditField_5.Value=string(C(2));
            app.EditField_6.Value=string(C(3));
        end

        % Button pushed function: PlotButton
        function PlotButtonPushed(app, event)
            syms x y z
            A(1)=str2sym(app.EditField.Value);
            A(2)=str2sym(app.EditField_2.Value);
            A(3)=str2sym(app.EditField_3.Value);
            C=[(diff(A(3),y)-diff(A(2),z)) -(diff(A(3),x)-
diff(A(1),z)) (diff(A(2),x)-diff(A(1),y))];
            xL=app.EditField_7.Value;
            xU=app.EditField_8.Value;
            yL=app.EditField_9.Value;
            yU=app.EditField_10.Value;
            zL=app.EditField_11.Value;
            zU=app.EditField_12.Value;
            s=app.EditField_13.Value;
            [X,Y,Z]=meshgrid(xL:s:xU,yL:s:yU,zL:s:zU);
            u=subs(C(1),{x,y,z},{X,Y,Z});
            v=subs(C(2),{x,y,z},{X,Y,Z});
            w=subs(C(3),{x,y,z},{X,Y,Z});
            quiver3(app.UIAxes,X,Y,Z,u,v,w);
        end

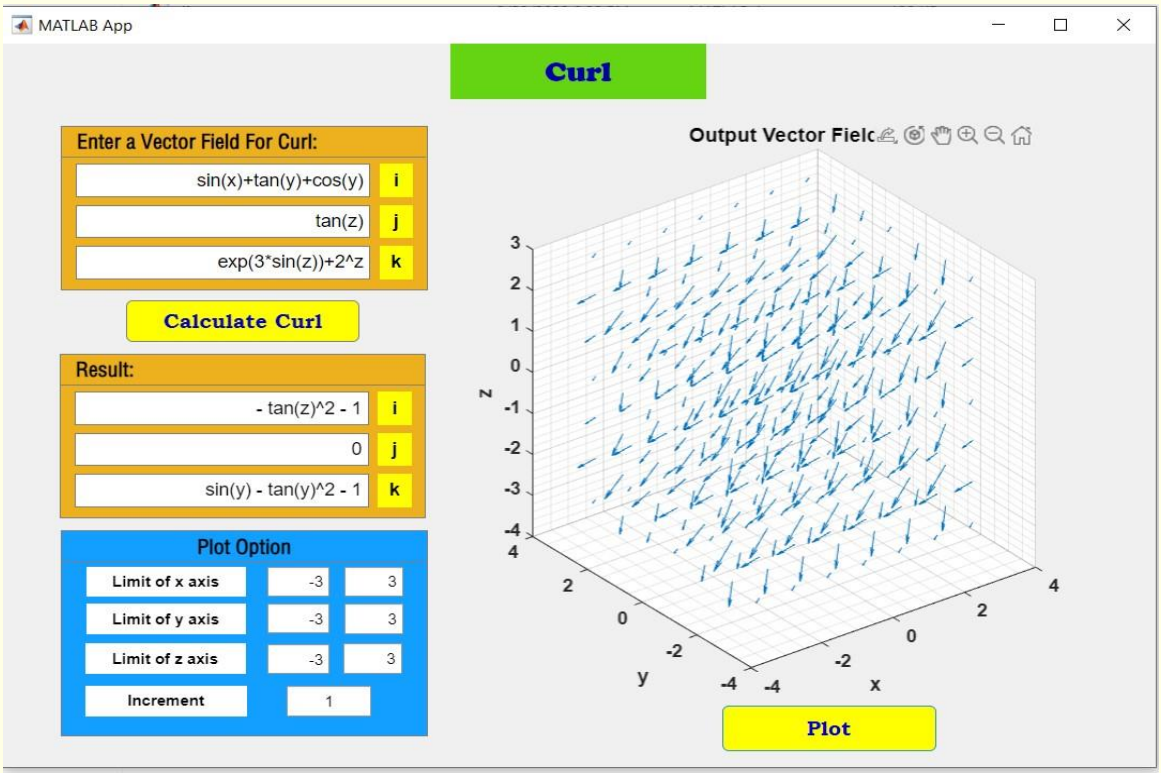
        % Callback function
        function HomeButtonPushed(app, event)
            Curl.app.callingapp=Home;
        end
    end
```

Sample results:

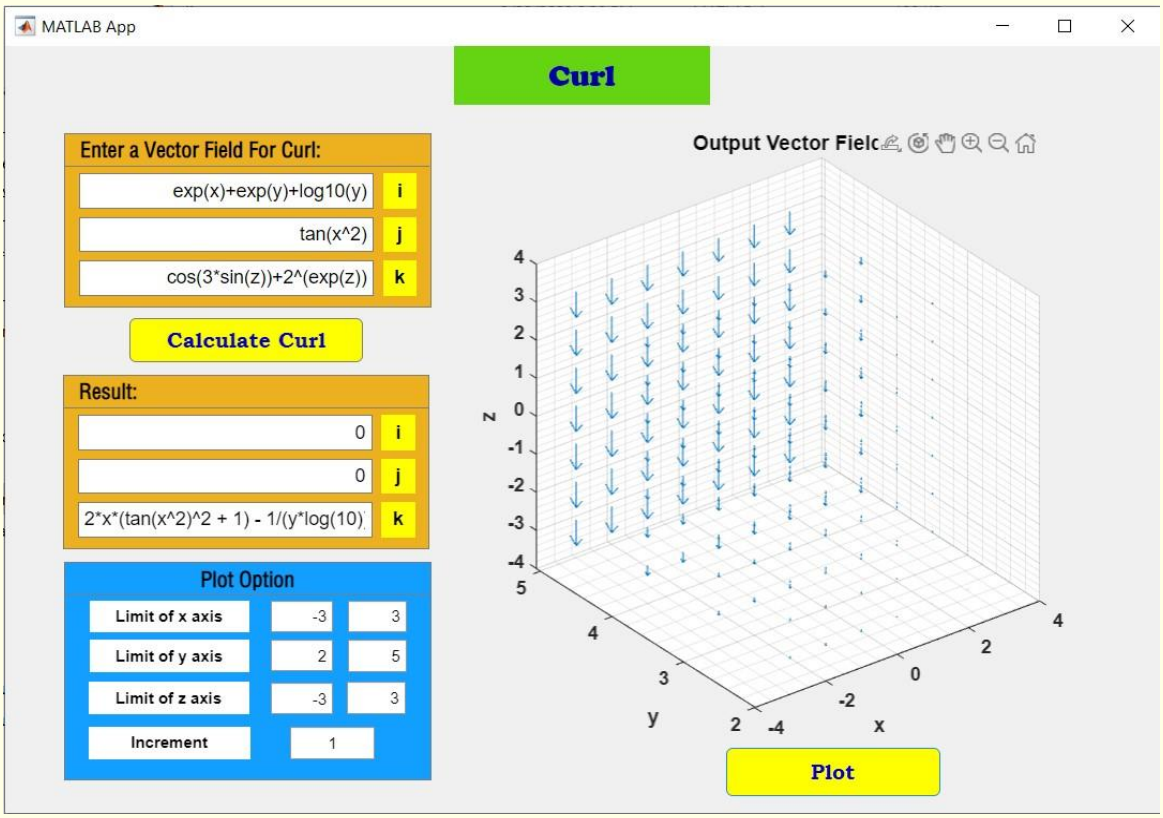
Case 1:



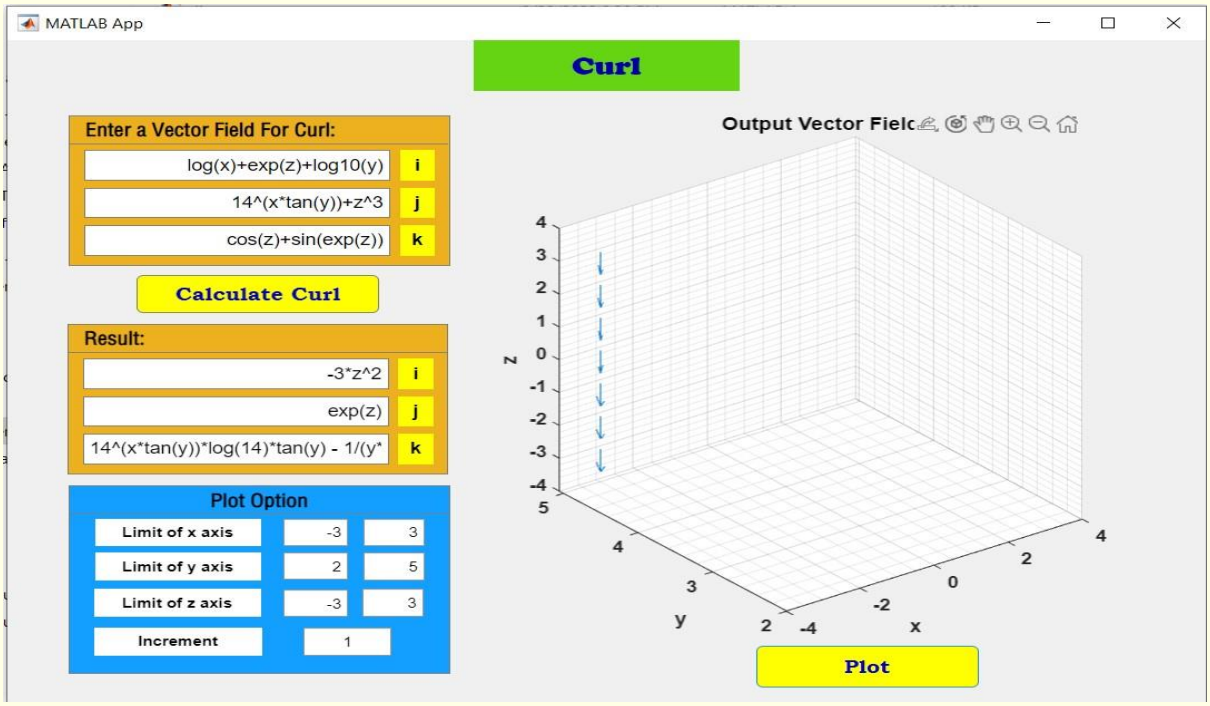
Case 2:



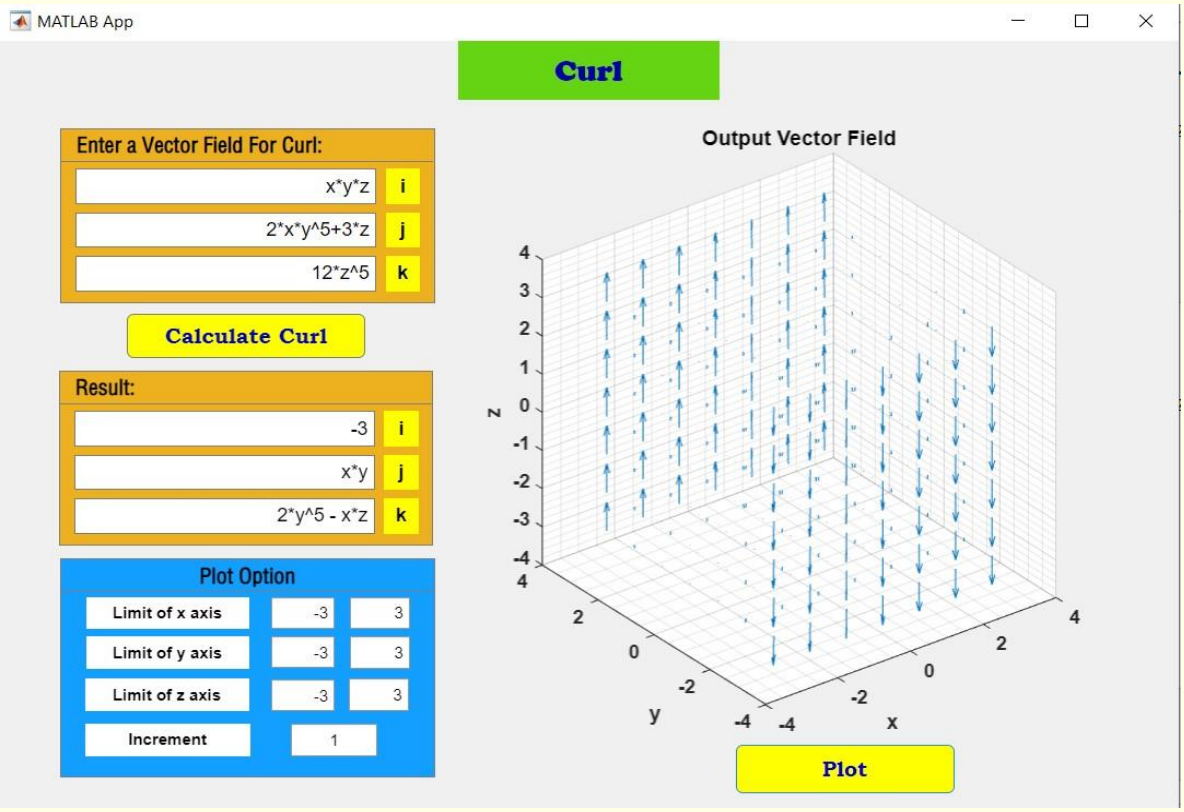
Case 3:


MATLAB App
Curl
Enter a Vector Field For Curl:
exp(x)+exp(y)+log10(y)
i
tan(x^2)
j
cos(3*sin(z))+2^(exp(z))
k
Calculate Curl
Result:
0
i
0
j
2*x*(tan(x^2)^2 + 1) - 1/(y*log(10)
k
Plot Option
Limit of x axis
-3
3
Limit of y axis
2
5
Limit of z axis
-3
3
Increment
1
Output Vector Fielc
Plot

Case 4:


MATLAB App
Curl
Enter a Vector Field For Curl:
log(x)+exp(z)+log10(y)
i
14^(x*tan(y))+z^3
j
cos(z)+sin(exp(z))
k
Calculate Curl
Result:
-3*z^2
i
exp(z)
j
14^(x*tan(y))*log(14)*tan(y) - 1/(y*
k
Plot Option
Limit of x axis
-3
3
Limit of y axis
2
5
Limit of z axis
-3
3
Increment
1
Output Vector Fielc
Plot

Case 5:



Limitations:

The written code is applicable only for explicit functions f(x,y,z). Besides the inputs must be in arithmetic form to show proper result, otherwise mathematical function will not work.

Future work:

The report on curl, divergence, and gradient in MATLAB provides a comprehensive understanding of these vector operators and their practical applications. However, there are several areas where the report can be expanded in the future to enhance its scope and usefulness.

One possible direction for future work is to explore the use of these operators in machine learning and data analysis. Vector calculus concepts such as gradient descent and divergence regularization are widely used in machine learning

algorithms, and a deeper understanding of these concepts can be helpful in developing more effective algorithms.

Another potential area of future work is to explore the use of these operators in computer graphics and image processing. The gradient operator is frequently used to detect edges and features in images, while the curl operator can be used to simulate fluid flow and turbulence in animations. Finally, future work could involve the development of more advanced MATLAB scripts and functions for calculating and visualizing vector fields using these operators. This includes the use of MATLAB's symbolic toolbox and Graphical User Interface (GUI) for more complex calculations and the development of interactive visualizations for exploring vector fields.

Discussion:

In this report, we have explored the concepts of curl, divergence, and gradient and their applications in MATLAB. We started with a theoretical background on these vector operators and then demonstrated their practical use in MATLAB through various examples and code snippets.

We tried to cover how these operators can be used to analyze and visualize vector fields, such as electric and magnetic fields, fluid flow, and temperature gradients.

Overall, this report provides a comprehensive understanding of curl, divergence, and gradient in MATLAB. We hope that this report serves as a valuable resource for researchers, students, and practitioners in various fields and inspires further exploration and development of these concepts in MATLAB .